A Replication and Analysis of the Pixel Stealing Attack

Jeremy Sheng, Joy Chen, Maddie Dreher, Diya Shrawagi
The University of Texas at Austin
{js224768,cc75925,mbd2325,ds54998}@eid.utexas.edu

Abstract

Modern web browsers enforce isolation between websites so that a malicious webpage should not be able to directly read the contents of another origin. However, side-channel attacks can sometimes infer information indirectly through shared browser and hardware resources. This project attempts to replicate the pixel stealing attack proposed by Andrysco et al. [1], which exploits hardware timing differences when computing on subnormal numbers. We first begin by investigating subnormal timing differences on different CPUs to determine the viability of the method and the potential attack surface. We then set up attacker and victim websites, confirm that the same-origin policy prevents direct access of the victim, then attempt to infer pixel colors from timing differences.

1 Introduction

Browsers are designed around the same-origin policy, which prevents one website from directly inspecting the contents of another website. Despite this isolation, webpages still share underlying resources such as the CPU, GPU, caches, and browser rendering system. Differences in how these resources behave while processing particular visual content can potentially leak information through a side channel.

There are various methods to leak pixel information, including exploitation of software timing differences [4], hardware timing differences [1], GPU compression [5], and microarchitectural cache state [3]. Our initial approach is to exploit hardware timing differences of subnormal floating point computation. We are also exploring GPU.zip as an alternative leaking method to subnormal timing.

2 Background

2.1 IEEE 754 Subnormal Floating-Point Numbers

The IEEE 754 standard for single-precision floating-point arithmetic allocates 32 bits into three fields: 1 sign bit (s), 8 exponent bits (e, with a bias of 127), and 23 significand bits (f). For standard normalized values ($e \in [1, 254]$), the representation assumes an implicit leading bit of 1:

$$x_{\text{norm}} = (-1)^s \times (1.f)_2 \times 2^{e-127} \tag{1}$$

The smallest positive normalized single-precision number occurs when $e = 1$ and $f = 0$, yielding $N_{\min} = 1.0_2 \times 2^{-126} \approx 1.175 \times 10^{-38}$.

When the exponent field is all zeros ($e = 0$), the value transitions to a subnormal (or denormal) representation. In subnormal mode, the implicit leading bit shifts from 1 to 0, and the effective exponent is fixed at -126:

$$x_{\text{sub}} = (-1)^s \times (0.f)_2 \times 2^{-126} \tag{2}$$

The smallest positive subnormal has bit pattern `0x00000001`, or $2^{-149} \approx 1.4 \times 10^{-45}$. Subnormals support gradual underflow, bridging the gap between 0.0 and $N_{\min}$.

2.2 Microarchitectural Timing Asynchrony

Modern x86 execution units are built for normalized operands, executing in 1 to 4 clock cycles ($\sim$ 1 ns); full hardware support for subnormals across every execution unit would cost significant silicon and power.

Consequently, when an execution pipeline encounters a subnormal operand or generates a subnormal output under standard IEEE 754 compliance (where Flush-to-Zero and Denormals-are-Zero modes are disabled), the fast execution datapath cannot process the value. The CPU flushes the out-of-order execution pipeline and delegates processing to internal microcode assist routines (`fp_assists`). Handling gradual underflow via microcode introduces a severe latency penalty of 80 to 200+ cycles ($\sim$ 30–70 ns) per operation, creating a data-dependent timing side-channel.

2.3 Pixel Stealing Attack

Browsers enforce the Same-Origin Policy (SOP): a page may embed a cross-origin image but cannot read its pixels directly, for example through HTML5 canvas extraction. Andrysco et al. [1] showed that subnormal timing can bypass the SOP and reconstruct cross-origin pixels:

1. **SVG Filter Processing:** A target cross-origin image is rendered inside an HTML5/SVG container. The attacker applies a custom SVG filter primitive (such as `feColorMatrix` or `feConvolveMatrix`) that scales normalized pixel color channel values $I \in [0.0, 1.0]$ by a minute scalar $\alpha \approx 10^{-42}$.
2. **Data-Dependent Latency:**
   - Non-Black Pixels ($I > 0$): Multiplying a non-zero color intensity by α yields a subnormal result ($\alpha I \approx 10^{-42}$). This repeatedly triggers microcode assist traps during rendering, significantly stalling the browser execution pipeline.
   - Black Pixels ($I = 0$): Multiplying zero by α yields exact zero (0.0), completing immediately on the fast hardware execution path without latency.

3. **Pixel Extraction:** By measuring frame render latencies using browser timing mechanisms (such as requestAnimationFrame or CSS repaint callbacks), the attacker classifies individual pixels as black or non-black based on rendering speed. Repeating this process across a target pixel grid progressively leaks private visual content.

3 Threat Model

We assume that an attacker controls a malicious website and convinces a user to visit it. The attacker does not have direct access to the victim website's DOM, memory, network traffic, or authentication information. Instead, the attacker can only execute ordinary browser code from the malicious origin and observe side-channel measurements exposed through shared browser or hardware behavior.

4 Our Approach

Our approach has three stages. First, we measure subnormal floating-point timing on a range of CPUs to find out which processors have a large enough penalty to attack. Second, we set up a cross-origin environment where an attacker page can isolate a single victim pixel and time how long it takes to render. Third, we will combine the two by applying an SVG filter that forces subnormal arithmetic and using the resulting timing differences to recover pixel colors. We have finished the first two stages, and the third is in progress.

4.1 Measuring Subnormal Timing

Since our intended method of leaking pixel information is to exploit subnormal timing differences as done in [1], we began by first gathering data on various CPUs. The attack only works if the target processor is noticeably slower on subnormal operands, so this step tells us which machines are worth targeting before we spend time on the browser side of the attack.

We developed two programs to evaluate the performance of floating-point addition and multiplication using both normal and subnormal operands. Separate implementations were written for the ARM and x86 ISAs, allowing us to measure execution times across a range of processors. Both programs are written in C, but the timed loops are written in inline assembly. This way, we know exactly which instructions are being measured, and the compiler cannot optimize the computation.

We ran these programs on eleven processors across both ISAs. By comparing the timing differences between normal and subnormal operations, we investigated how these variations depend on the underlying ISA and processor architecture, identifying potential targets for our subsequent attack. The results are discussed in detail in Section 5.

4.2 Setting Up the Attack

We also began setting up the attack. Our setup uses a small Python server that hosts two origins on the local machine, one for the attacker and one for the victim. The two origins run on different ports, so the browser treats them as cross-origin and enforces the same-origin policy between them. As described in our threat model, the victim page shows an 8x8 image of black and white pixels.

The attacker page loads the victim page in a cross-origin `<iframe>`. Before measuring anything, it tries to read the victim's DOM directly and checks that the browser blocks it with a `SecurityError`. This confirms that anything the attacker learns later has to come from the side channel and not from a mistake in our setup.

To look at one pixel at a time, the attacker page puts the iframe inside a 1x1 CSS pixel container that hides anything outside of it, and shifts the iframe so that only the target pixel shows through. It then scales the container up with a CSS `scale()` transform, so the single pixel covers up to a 300x300 area on the screen. This idea comes from Stone [4] and Andrysco et al. [1], and it is a key part of the attack. A penalty of tens of nanoseconds per operation is much too small to see from JavaScript, but once a filter is applied, every pixel in the enlarged area pays that penalty, which adds up to something large enough to measure with frame timing.

To measure rendering time, the attacker page records the time between two `requestAnimationFrame` callbacks using `performance.now()`, which follows the approach of Stone [4] and Andrysco et al. [1]. The page has two modes. The calibration mode times black and white patches that the attacker controls, giving us reference timings for each color. The scan mode goes through all 64 victim pixels and takes a set number of samples for each one.

4.3 Next Steps

Right now, the attacker page does not apply any SVG filters, and as such the rendering time does not depend on the pixel's color and we do not see any timing differences. The last stage of our approach is to apply a filter that scales pixel color values into the subnormal range, as described in Section 2, so that non-black pixels are slowed down by the subnormal penalty and black pixels are not. We will then use the calibration mode to pick a timing threshold between the two colors, classify each victim pixel using that threshold, and check our accuracy against the ground truth. With the hardware penalty and the filter's arithmetic both confirmed (Section 5.2), what remains is purely on the browser side: we still need to find a browser setup where the filter runs in floating point on the CPU of a vulnerable processor, and we will have to work around the reduced timer precision in modern browsers.

5 Evaluation

5.1 Timing Data

We measured floating-point add and multiply timing across a range of ARM and x86 processors, from personal laptops to server machines such as the ECE LRC servers. The results are summarized in Figure 1.

On the Apple M1 and M5 chips, there is no significant timing difference between normal and subnormal operands for both add and multiply. This is expected, since Apple silicon has full hardware support for subnormal floating-point operations and can execute at full speed.

On the six Intel chips, including the Intel Xeon 5218 used by the ECE LRC machines, we see significant deltas between normal and subnormal execution. This is the slow microcode assist discussed in Section 2, where the pipeline is flushed to handle the subnormal value. The penalty appears only for multiplication; subnormal addition runs at normal speed on every Intel chip we tested, so these chips evidently handle subnormal adds in hardware but fall back to the microcode assist for multiplies. For our attack, this means the SVG filter needs to create subnormal values through multiplication, which is how the `feColorMatrix` and `feConvolveMatrix` primitives scale pixel values.

The AMD EPYC Milan processor, which uses the Zen 3 microarchitecture, behaves very differently from the Intel chips. There is no penalty for subnormal addition (0.87 vs. 0.85 ns/op), and subnormal multiplication is only slightly slower than normal multiplication (1.14 vs. 0.90 ns/op). This difference of about 0.24 ns is roughly one clock cycle, about 100 times smaller than the 24–109 ns penalties we measured on the Intel chips. A penalty this small suggests Zen 3 handles subnormals in its normal floating-point hardware rather than through a microcode assist. We can measure it in a tight native loop, but it is unlikely to survive the noise of browser rendering and JavaScript's coarse timers.

The NVIDIA Grace processor, which is built from Arm Neoverse V2 cores [2], shows no difference at all between normal and subnormal operands for either operation (0.61 ns/op for addition and 0.92 ns/op for multiplication in both cases). Like the Apple chips, these Arm cores handle subnormals at full speed. None of the three Arm chips we tested showed a penalty, so the attack is unlikely to work on Arm machines.

Given these results, we will target the Intel Xeon 5218: it shows a dramatic penalty and is easy to access through the ECE LRC servers, while the Apple, AMD, and NVIDIA parts show little to none.

5.2 Filter-Level Timing Gap

The data in Section 5.1 times single instructions in a tight loop. To check that the gap also appears in the real filter computation, we wrote a small C program that runs the same math the SVG filter does (an `feColorMatrix` scale followed by repeated `feConvolveMatrix` passes), but outside the browser. It multiplies each pixel by a tiny factor $\alpha \approx 8.2 \times 10^{-42}$, which turns a white pixel (1.0) into a subnormal number and leaves a black pixel (0.0) at zero. It then runs 50 passes of a 25-tap averaging filter, about 10^7 multiplications. We time an all-white buffer against an all-black buffer on the Intel Xeon 5218. White is about 16.5 times slower (50.2 vs. 3.0 ns per multiplication), matching the penalty from Section 5.1. Turning on Flush-to-Zero, which makes the hardware treat subnormals as zero, removes the gap (1.00 times) and drops the white result from a tiny nonzero value to exactly zero. This shows the slowdown comes from the subnormal values and nothing else.

This is the key piece the attack is built on: the filter's math is more than ten times slower on a white pixel than a black one. It does not yet mean the attack works in a browser, which must run the filter on the CPU rather than the GPU, keep the subnormals instead of flushing them to zero, and leave a timing gap big enough to see through browsers' coarse post-Spectre timers (Section 5.3). Testing that on the Xeon 5218 is our next step.

5.3 The Attack Setup

We host a victim and an attacker website on separate local ports via a small Python server. The victim shows an 8x8 grid of 32 black and 32 white pixels in random order, fixed until the server restarts, which gives us a known ground truth to compare against.

The attack website loads the victim in a cross-origin iframe and confirms that the same-origin policy blocks it from reading the victim's DOM. It can then select a single pixel, enlarge it, and measure frame render times. Both calibration and scan modes work end to end, but since no SVG filter is applied yet, the frame times do not depend on pixel color and recover nothing.

One concern with the attack setup is timer precision. After Spectre, browsers reduced the precision of `performance.now()` and added jitter, and frame times are also limited by the display's refresh rate. We expect that we will need a large scale factor to make the subnormal penalty a large part of each frame's rendering time, and that we will need many samples per pixel to reliably tell the two colors apart.

6 Related Work

Stone's *Pixel Perfect Timing Attacks with HTML5* [4] demonstrated the feasibility of cross-origin pixel stealing through browser rendering behavior. By exploiting software timing differences in the HTML5 `requestAnimationFrame` API, the attack measures how quickly CSS and SVG filters process pixels within a cross-origin iframe, allowing an attacker to infer image contents. This work established browser rendering operations as an unintended information channel and

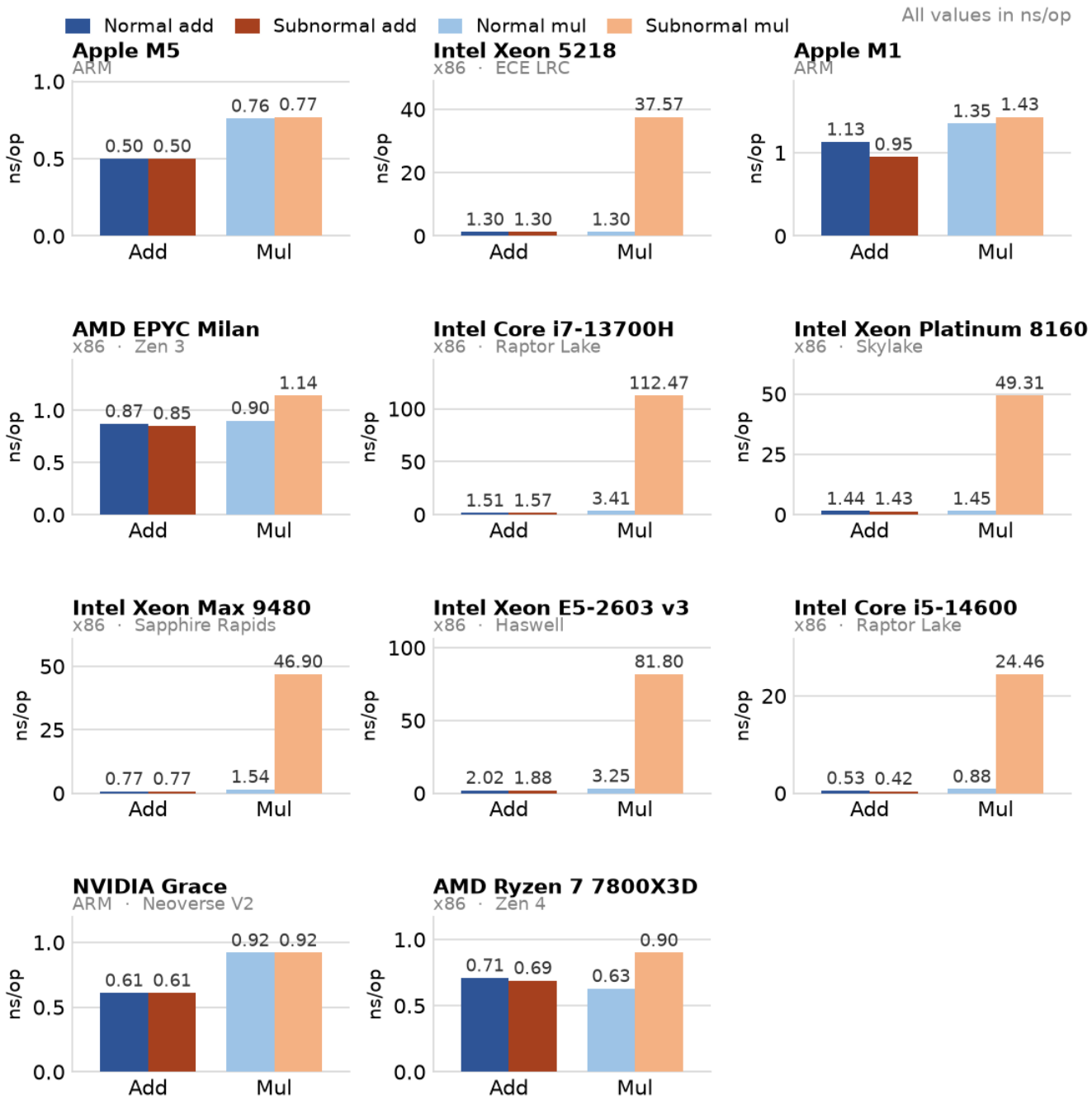


Figure 1. Time to perform floating-point instructions, measured in ns/op. Note the y-axis for each graph is different.

laid the foundation for subsequent SVG-based timing attacks. The following works all build upon this paper.

Andrysco et al. [1] extended this approach by identifying hardware-level timing differences in floating-point operations. They demonstrated that arithmetic involving subnormal floating-point numbers can execute significantly slower than operations on normal numbers. By constructing SVG filters that trigger these operations, attackers can distinguish pixel values based on execution time. Unlike Stone's approach, which exploits browser rendering behavior, this attack leverages CPU-level floating-point performance characteristics, demonstrating how hardware behavior can undermine seemingly constant-time software.

Wang et al.'s *GPU.zip* [5] expanded side-channel research to integrated GPU architectures by exploiting hardware-based graphical data compression. The authors demonstrated that undocumented, lossless compression mechanisms can introduce data-dependent differences in DRAM traffic and rendering time. By constructing carefully designed SVG filters that produce highly compressible pixel patterns, attackers can infer underlying pixel values. This work showed that hardware optimizations intended to improve performance can themselves introduce information leakage, extending earlier attacks beyond CPU execution timing.

O'Connell et al.'s *Pixel Thief* [3] further advanced pixel-stealing attacks by exploiting CPU cache-state changes rather than relying directly on elapsed execution time. By monitoring the cache footprint left by SVG filter execution, the attack extracts pixel information despite modern browser timing mitigations, including reduced timer resolution and jitter.

Collectively, these works demonstrate the evolution of browser-based side-channel attacks from software-level rendering measurements to hardware-level information leakage. Stone established the feasibility of timing-based pixel extraction, Andrysco et al. identified CPU floating-point timing differences, GPU.zip exposed leakage through GPU memory compression, and Pixel Thief demonstrated cache-based extraction. Together, they highlight the limitations of software-level timing mitigations and the broader security implications of data-dependent hardware behavior.

7 Conclusion

In this midpoint report, we described our progress so far in replicating the pixel stealing attack by Andrysco et al. [1]. Measuring floating-point timing across eleven ARM and x86 processors, we found that the subnormal multiply penalty is still present on modern Intel processors, roughly 30 times slower than normal multiplication, but mostly gone on AMD Zen 3, Apple silicon, and NVIDIA Grace, so we chose the Intel Xeon 5218 as our target.

We also built the setup for the end-to-end attack: a cross-origin victim page with a known secret image, and an attacker page that confirms the same-origin policy is enforced, enlarges individual victim pixels, and measures frame render times. For the rest of the project, we plan to drive the subnormal filter through a browser that runs it on the CPU and measure how accurately we can recover the secret image. Whether or not the attack works on a modern browser, we expect our results to show how hardware timing behavior and browser defenses have changed since the original attack was published.

References

[1] Andrysco, M., Kohlbrenner, D., Mowery, K., Jhala, R., Lerner, S., and Shacham, H. On subnormal floating point and abnormal timing. In *2015 IEEE Symposium on Security and Privacy* (May 2015), IEEE.

[2] NVIDIA Corporation. NVIDIA Grace CPU Superchip. https://www.nvidia.com/en-us/data-center/grace-cpu-superchip/, 2023.

[3] O'Connell, S., Sour, L. A., Magen, R., Genkin, D., Oren, Y., Shacham, H., and Yarom, Y. Pixel thief: Exploiting SVG filter leakage in firefox and chrome. In *33rd USENIX Security Symposium (USENIX Security 24)* (2024), USENIX Association, pp. 3331–3348.

[4] Stone, P. Pixel perfect timing attacks with html5. Black Hat USA, July 2013.

[5] Wang, Y., Paccagnella, R., Gang, Z., Vasquez, W., Kohlbrenner, D., Shacham, H., and Fletcher, C. GPU.zip: On the side-channel implications of hardware-based graphical data compression. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), IEEE.